TASK 9 – Implement and Test the Billing Program 4 Marks

Bloom's Level: K3 – Apply

Implement the complete billing program and test it.

Test

:• Normal bills in each slab

• Boundary cases: exactly 100 units, exactly 200 units, zero units

• A meter that has rolled over from 65535 to 0

• Invalid input: letters, a negative reading, a current reading less than the previous one

without roll-over • A small set of 5 bills whose total you calculate by hand

Alternative Concept and Complexity Analysis

If the slab limits and rates were stored in two arrays and the bill computed by a loop over the slabs, instead of fixed if statements, what would the solution be? Analyse the time per bill for s

slabs (O(s) versus a fixed number of tests) and the extra space, and explain which design is easier to change.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

TASK 9 – Implement and Test the Billing Program

1. Aim

To implement a complete electricity billing program in C, calculate charges using slab rates, handle meter rollover, validate input, and test boundary cases.

2. Billing Rules

The program uses the following tariff values:

Parameter	**Value**
Fixed charge	₹50.00
First slab	0–100 units at ₹0 per unit
Second slab	101–200 units at ₹2.25 per unit

Third slab	Above 200 units at ₹4.50 per unit

The total bill is calculated by adding the fixed charge and the energy charges from the applicable slabs. Money is stored in integer paise to avoid floating-point rounding errors.

3. Input Validation and Meter Rollover

The program checks whether the meter readings are valid non-negative integers within the range 0–65535.

If the previous reading is near the maximum value and the current reading is near zero, the program treats it as a meter rollover. For example, a change from 65535 to 0 represents one unit consumed.

If the current reading is less than the previous reading without satisfying the rollover condition, the program rejects the readings as invalid.

4. Test Cases

Test	**Previous**	**Current**	**Expected result**
Zero units	100	100	₹50.00
First slab	100	150	₹162.50
Exactly 100 units	100	200	₹50.00
Exactly 200 units	100	300	₹275.00
Third slab	100	350	₹950.00
Meter rollover	65535	0	1 unit; ₹50.00
Invalid letters	abc	—	Invalid input message
Negative reading	-10	100	Rejected
Invalid reading order	500	400	Rejected

5. Total of Five Bills

Consider five bills with consumption values of 0, 100, 150, 200 and 350 units.

Units	**Bill**
0	₹50.00
100	₹50.00

150	₹162.50
200	₹275.00
350	₹950.00
Total	**₹1,487.50**

The total is calculated by adding all five bills manually and comparing it with the program's result.

6. Alternative Concept: Arrays and Loops

Instead of using separate if statements, slab limits and rates can be stored in arrays. A loop can process the applicable slabs.

For (s) slabs:

- Fixed if statements take constant time, (O(1)), when the number of slabs is fixed.
- A loop over the slab arrays takes (O(s)) time per bill.
- The array-based approach requires (O(s)) extra space for the slab limits and rates.
- Fixed if statements are simple for a small, unchanging tariff.
- Arrays and loops are easier to maintain when tariff limits or the number of slabs change.

Code :

```
#include <stdio.h>
#define MAX_READING 65535
#define FIXED_CHARGE 5000
long calculate_bill(long units)
{
  long bill = FIXED_CHARGE;
  if (units > 200)
    bill += (units - 200) * 450L;
  if (units > 100) {
    long slab2 = units < 200 ? units - 100 : 100;
```

```
        bill += slab2 * 225L;
    }
    return bill;
}
void test_bill(const char *name, long previous, long current)
{
    long units, bill;
    printf("\nTest: %s\n", name);
    printf("Previous: %ld, Current: %ld\n", previous, current);
    if (previous < 0 || current < 0 ||
        previous > MAX_READING || current > MAX_READING) {
        printf("Result: Invalid meter reading\n");
        return;
    }
    if (current < previous) {
        if (previous >= 60000 && current <= 1000)
            units = MAX_READING - previous + 1 + current;
        else {
            printf("Result: Invalid readings (no rollover)\n");
            return;
        }
    } else {
        units = current - previous;
    }
    bill = calculate_bill(units);

    printf("Units consumed: %ld\n", units);
    printf("Bill: Rs %ld.%02ld\n", bill / 100, bill % 100);
```

```
}

int main(void)
{
    long readings[][2] = {
        {100, 100},
        {100, 200},
        {100, 250},
        {100, 300},
        {100, 450},
        {65535, 0},
        {500, 400},
        {-10, 100},
        {65536, 100}
    };

    const char *names[] = {
        "Zero units",
        "Exactly 100 units",
        "150 units",
        "Exactly 200 units",
        "350 units",
        "Meter rollover",
        "Invalid reading order",
        "Negative reading",
        "Reading above maximum"
    };
    int i;
```

```
printf("===== BILLING TESTS =====\n");
for (i = 0; i < 9; i++)
   test_bill(names[i], readings[i][0], readings[i][1]);
printf("\n===== INVALID TEXT INPUT TEST =====\n");

{
   char input[] = "abc";
   long value;
   char *end;
   value = 0;
   end = input;
   while (*end == ' ')
      end++;

   if (*end < '0' || *end > '9')
      printf("Input: abc -> Invalid input\n");
}

printf("\n===== FIVE BILL TOTAL =====\n");
{
   long bills[] = {0, 100, 150, 200, 350};
   long total = 0;
   int j;
   for (j = 0; j < 5; j++) {
      long bill = calculate_bill(bills[j]);
      printf("%ld units: Rs %ld.%02ld\n",
           bills[j], bill / 100, bill % 100);
      total += bill;
```

```
        }
        printf("Total: Rs %ld.%02ld\n", total / 100, total % 100);
    }
    return 0;
}
```

Output:



Conclusion

The completed billing program calculates charges, handles meter rollover, validates invalid input, and tests normal and boundary cases. Comparing expected and actual results helps verify the correctness of the program.